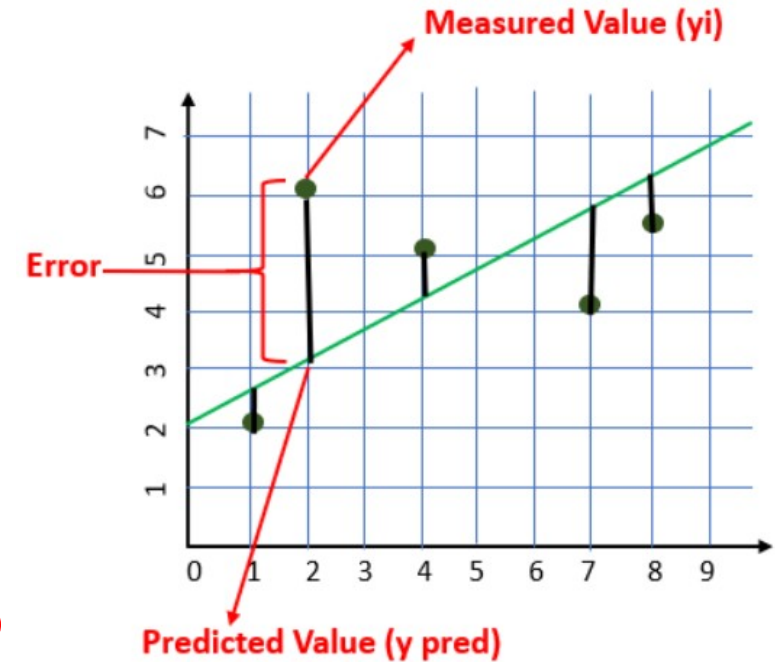


Gradient Descent

Cost/Loss Function (Recap)

- Machines learn by considering loss/cost function.
- Loss/cost functions measure how far an estimated value is from its true value.
- If predictions deviates too much from actual results, loss function will be a very large number.
- Gradually, with the help of some optimization function, loss function learns to reduce the error in prediction.



Contd...

- The cost is the error in our predicted value. We will use the Mean Squared Error function to calculate the cost.

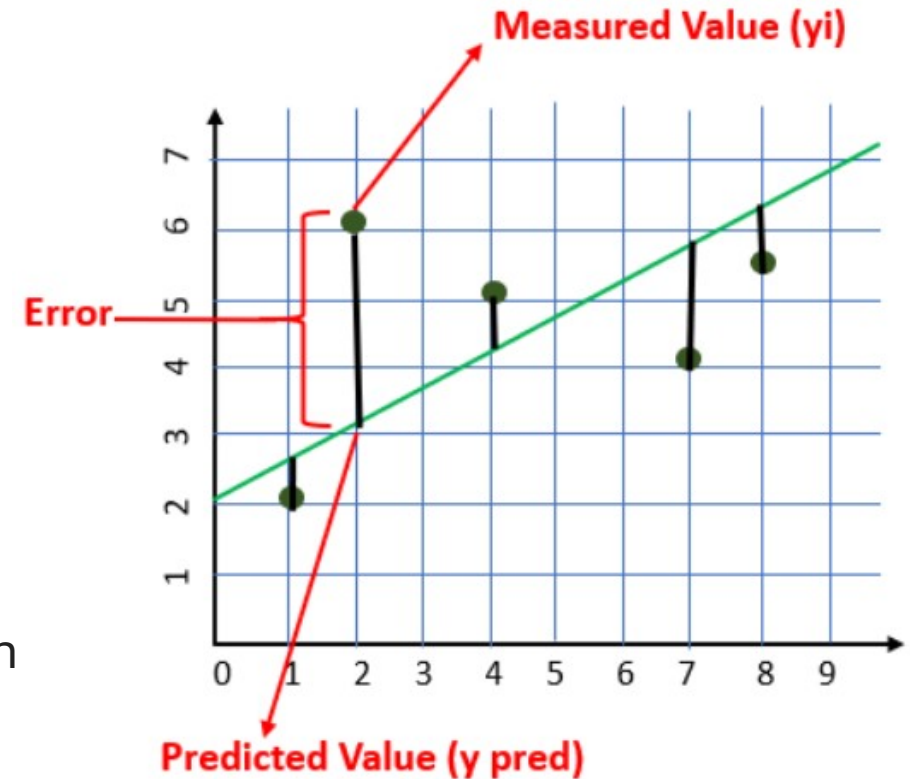
$$\text{Cost Function(MSE)} = \frac{1}{n} \sum_{i=0}^n (y_i - y_{i \text{ pred}})^2$$

Replace $y_{i \text{ pred}}$ with $mx_i + c$

$$\text{Cost Function(MSE)} = \frac{1}{n} \sum_{i=0}^n (y_i - (mx_i + c))^2$$

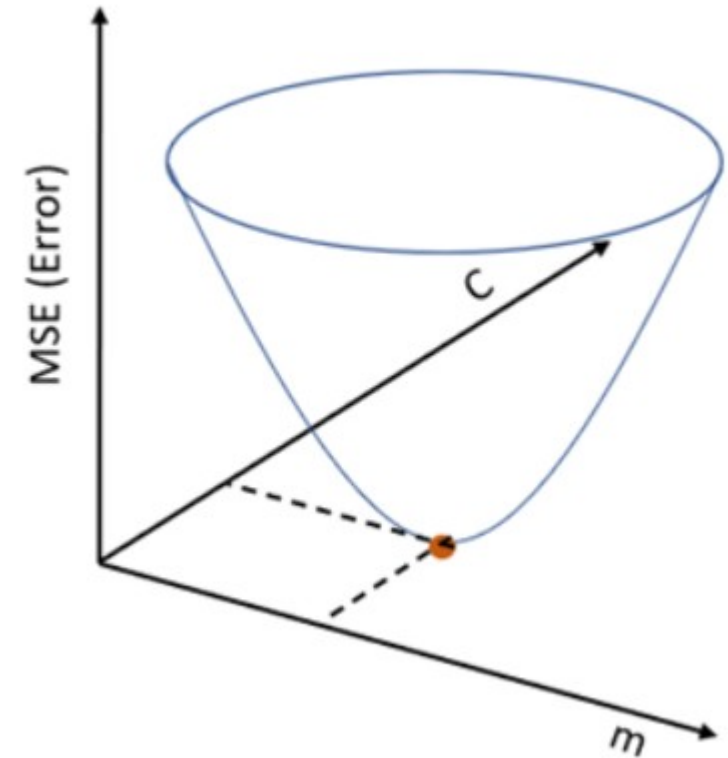
Our goal is to minimize the cost as much as possible in order to find the best fit line.

Try all the permutation and combination of m and c (inefficient way) to find the best-fit line?.



Gradient Descent Algorithm

- The **Gradient Descent algorithm** is an optimization technique used to minimize the error (loss/cost) of a machine learning model by iteratively updating the model parameters.
- It finds the best-fit solution (example: line), with minimal loss (error) for a given training dataset in a smaller number of iterations.
 - If we plot the weight (m and c) against loss (error-MSE), it will acquire a bowl shape
 - For some combination of the weights, we will get the least Error (MSE).
 - That combination of weights will give us our best fit line.



An analogy for understanding gradient descent

- Imagine you are standing on a **mountain** and want to reach the **lowest point (valley)**, but you are blindfolded.
 - You feel the **slope** under your feet
 - You take a **small step downhill**
 - Repeat until the ground becomes flat
- That is what gradient descent does.
 - **Mountain** → Loss function
 - **Your position** → Model parameters (weights, for example m and c in the case of a line)
 - **Slope** → Gradient
 - **Step size** → Learning rate



- **What is a gradient?**

- **Gradient = rate of change of the cost with respect to the model parameters (weights).**

- Mathematically:

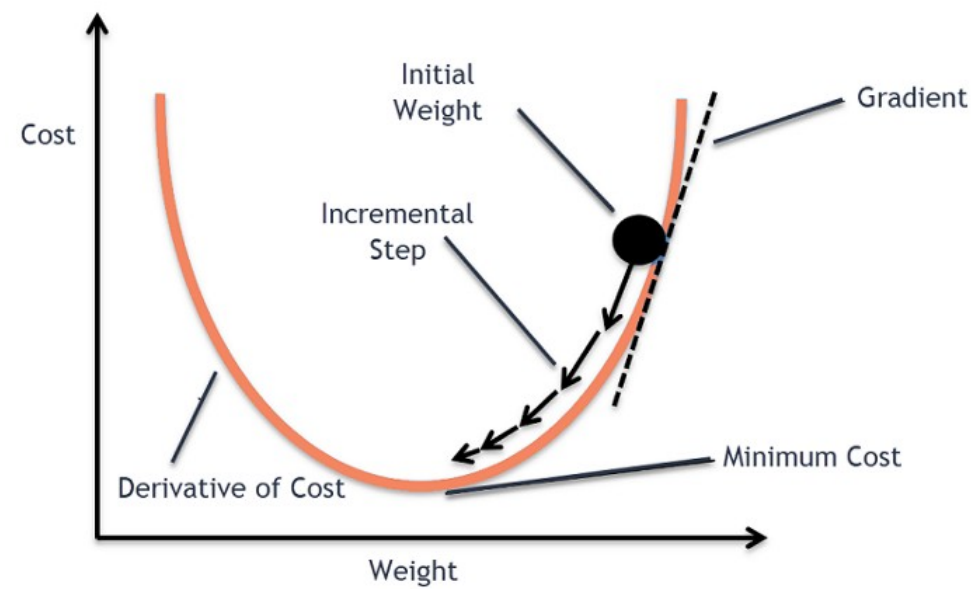
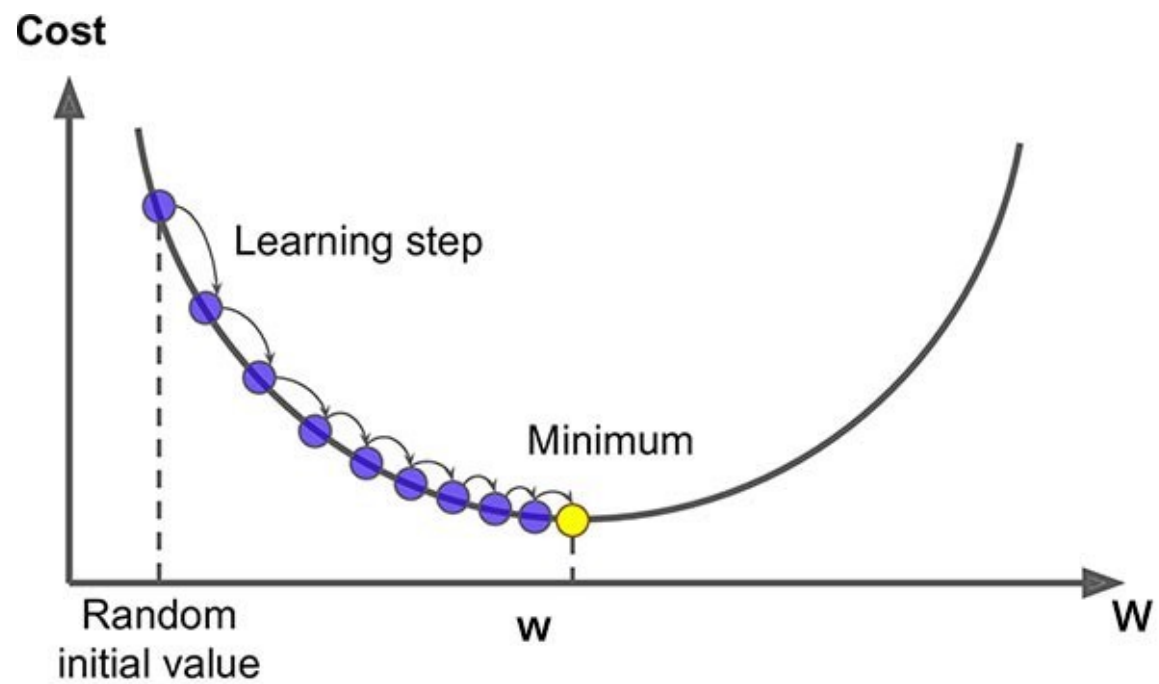
$$\text{Gradient} = \frac{\partial \text{Cost}}{\partial \text{Weight}}$$

- A **gradient** tells us:

- **Direction** to move
- **How steep** the slope is

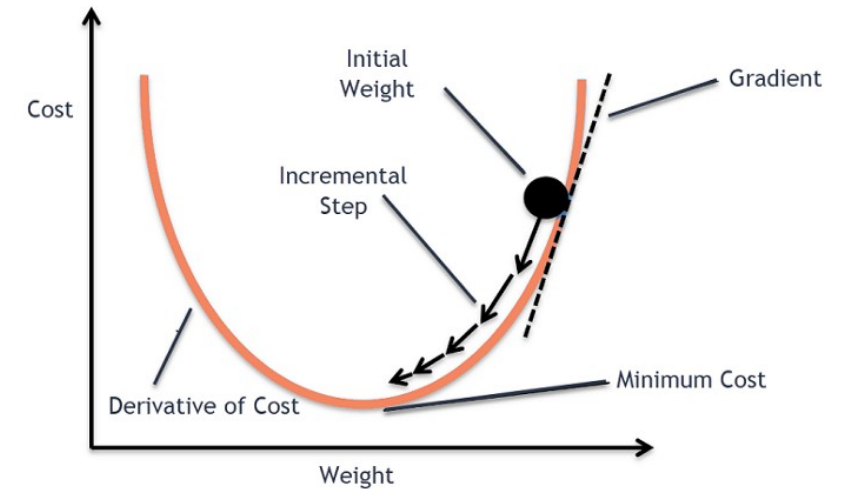
- Mathematically, it is the **derivative of the loss function** with respect to model parameters.

- Positive gradient → move left
- Negative gradient → move right



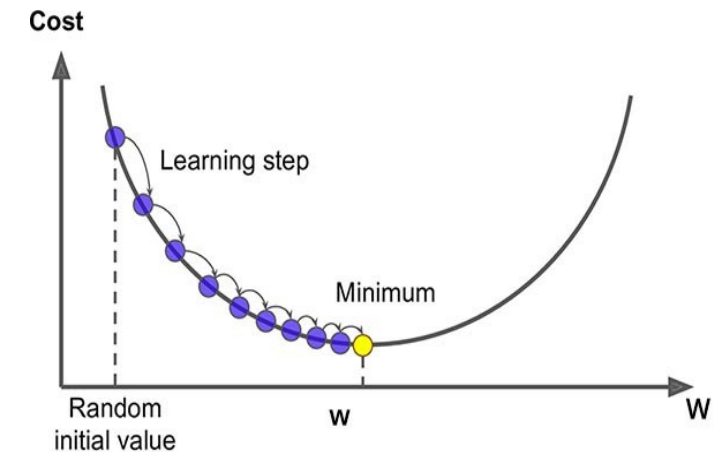
Why “descent”?

- The **gradient** points in the direction of **steepest increase** of the loss function
- So, to **minimize** the loss, we move in the **opposite direction of the gradient** to reduce error.
- $\text{New weight} = \text{Old weight} - \text{Learning Rate} \times \text{Gradient}$



Learning Rate (very important!)

- The **learning rate** (η or L or α) controls how big each step is.
 - Too small $\rightarrow$ training is very slow
 - Too large $\rightarrow$ may overshoot the minimum
 - Just right $\rightarrow$ fast and stable learning



• Algorithm

- Initialize weights randomly
- Compute prediction
- Calculate loss (error)
- Compute gradient (slope)
- Update weights
- Repeat until loss is minimum

Step 1: Initialize the Parameters

- Choose initial values for the model parameters.
- Initialize:
 - Slope m
 - Intercept c

Example:

- $m = 0$
- $c = 0$

Step 2: Select the Learning Rate

Choose a learning rate (α).

Example:

- $\alpha = 0.01$

Step 3: Make Predictions

For linear regression:

$$\hat{y} = mx + c$$

Step 4: Compute the Error (Cost Function)

Using Mean Squared Error (MSE):

$$E(m, c) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- n = number of training samples
- y_i = actual value
- $\hat{y}_i$ = predicted value

Step 5: Compute the Gradients

Gradient with respect to m :

$$\frac{\partial E}{\partial m} = \frac{-2}{n} \sum_{i=0}^n x_i (y_i - y_{i \text{ pred}})$$

Gradient with respect to c :

$$\frac{\partial E}{\partial c} = \frac{-2}{n} \sum_{i=0}^n (y_i - y_{i \text{ pred}})$$

Step 6: Update the Parameters

Update the slope:

$$m = m - \alpha \frac{\partial E}{\partial m}$$

Update the intercept:

$$c = c - \alpha \frac{\partial E}{\partial c}$$

Step 7: Repeat

Repeat Steps 3–6 until:

- the error E stops decreasing,
- the gradients become very small,
- or the maximum number of iterations is reached.

Step 8: Obtain the Optimal Parameters

The final values of m and c minimize the error and define the best-fit regression line.

Computation of gradient

Assume that the model is given by the equation:

$$Y_{pred} = mx_i + c$$

Assume that Mean Squared Error function is used to calculate the cost (Error)

$$Cost\ Function(MSE) = \frac{1}{n} \sum_{i=0}^n (y_i - y_{i\ pred})^2$$

Replace $y_{i\ pred}$ with $mx_i + c$

$$Cost\ Function(MSE) = \frac{1}{n} \sum_{i=0}^n (y_i - (mx_i + c))^2$$

Hear, Y_i is the actual value and Y_{pred} is the predicted value

1. To compute the gradient, calculate the partial derivative of the Cost function with respect to m . Let partial derivative of the Cost function with respect to m be D_m (With little change in m how much Cost function changes).

$$\begin{aligned} D_m &= \frac{\partial(\text{Cost Function})}{\partial m} = \frac{\partial}{\partial m} \left(\frac{1}{n} \sum_{i=0}^n (y_i - y_{i \text{ pred}})^2 \right) \\ &= \frac{1}{n} \frac{\partial}{\partial m} \left(\sum_{i=0}^n (y_i - (mx_i + c))^2 \right) \\ &= \frac{1}{n} \frac{\partial}{\partial m} \left(\sum_{i=0}^n (y_i^2 + m^2 x_i^2 + c^2 + 2mx_i c - 2y_i m x_i - 2y_i c) \right) \\ &= \frac{-2}{n} \sum_{i=0}^n x_i (y_i - (mx_i + c)) \\ &= \frac{-2}{n} \sum_{i=0}^n x_i (y_i - y_{i \text{ pred}}) \end{aligned}$$

2. Similarly, find the partial derivative with respect to c. Let partial derivative of the Cost function with respect to c be D_c (With little change in c how much Cost function changes).

$$\begin{aligned} D_c &= \frac{\partial(\text{Cost Function})}{\partial c} = \frac{\partial}{\partial c} \left(\frac{1}{n} \sum_{i=0}^n (y_i - y_{i \text{ pred}})^2 \right) \\ &= \frac{1}{n} \frac{\partial}{\partial c} \left(\sum_{i=0}^n (y_i - (mx_i + c))^2 \right) \\ &= \frac{1}{n} \frac{\partial}{\partial c} \left(\sum_{i=0}^n (y_i^2 + m^2 x_i^2 + c^2 + 2mx_i c - 2y_i mx_i - 2y_i c) \right) \\ &= \frac{-2}{n} \sum_{i=0}^n (y_i - (mx_i + c)) \\ &= \frac{-2}{n} \sum_{i=0}^n (y_i - y_{i \text{ pred}}) \end{aligned}$$

3. Now update the current values of m and c using the following equation:

$$m = m - LD_m$$

$$c = c - LDc$$

Where, L is the Learning Rate. L is also denoted as α OR η

4. Repeat this process until our Cost function is very small (ideally 0).

- Significance of Learning rate
 - The **learning rate (η or L or α)** controls how big each step is.
 - Too small $\rightarrow$ training is very slow
 - Too large $\rightarrow$ may overshoot the minimum
 - Just right $\rightarrow$ fast and stable learning
 - The choice of correct learning rate is very important as it ensures that Gradient Descent converges in a reasonable time. :
 - If we choose L or α to be **very large**, Gradient Descent can overshoot the minimum. It may fail to converge or even diverge.
 - If we choose α to be very small, Gradient Descent will take small steps to reach local minima and will take a longer time to reach minima.

